字节序处理和消息队列的控制

Posted on 2023-05-13 18:48:01 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2PjeNxtYmEymWtseA7kd1jeCDYT)

字节序的问题

在计算机网络中，由于不同的计算机使用的 CPU 架构和字节顺序可能不同，因此在传输数据时需要对数据的字节序进行统一，以保证数据能够正常传输和解析。这就是网络字节序的作用。

具体来说，计算机内部存储数据的方式有两种：大端序（Big-Endian）和小端序（Little-Endian）。在大端序中，高位字节存储在低地址处，而低位字节存储在高地址处；在小端序中，高位字节存储在高地址处，而低位字节存储在低地址处。

在网络通信过程中，通常使用的是大端序。这是因为早期的网络硬件大多采用了 Motorola 处理器，而 Motorola 处理器使用的是大端序。此外，大多数网络协议规定了网络字节序必须为大端序。

因此，在进行网络编程时，需要将主机字节序转换为网络字节序，也就是将数据从本地字节序转换为大端序。可以使用诸如 htonl、htons、ntohl 和 ntohs 等函数来实现字节序转换操作。

综上所述，网络字节序的主要作用是统一不同计算机间的数据表示方式，以保证数据在网络中的正确传输和解析。

如何区分本机字节序

如何区分本机字节序，可以通过判断低地址存储的数据是否为低字节数据，如果是则为小端，否则为大端，下面写一段代码讲述这个逻辑

```cpp
#include <iostream>

using namespace std;

// 判断当前系统的字节序是大端序还是小端序
bool is_big_endian() {
    int num = 1;
    if (*(char*)&num == 1) {
        // 当前系统为小端序
        return false;
    } else {
        // 当前系统为大端序
        return true;
    }
}

int main() {
    int num = 0x12345678;
    char* p = (char*)&num;

    cout << "原始数据：" << hex << num << endl;

    if (is_big_endian()) {
        cout << "当前系统为大端序" << endl;
        cout << "字节序为：";
        for (int i = 0; i < sizeof(num); i++) {
            cout << hex << (int)*(p + i) << " ";
        }
        cout << endl;
    } else {
        cout << "当前系统为小端序" << endl;
        cout << "字节序为：";
        for (int i = sizeof(num) - 1; i >= 0; i--) {
            cout << hex << (int)*(p + i) << " ";
        }
        cout << endl;
    }

    return 0;
}
```

在上述代码中，使用了一个 is_big_endian() 函数来判断当前系统的字节序是否为大端序。该

函数通过创建一个整型变量 num，并将其最低位设置为 1，然后通过指针强制转换成字符指针，判断第一个字节是否为 1 来判断当前系统的字节序。

在 main 函数中，定义了一个整型变量 num，并将其初始化为 0x12345678。接着，使用 char* 类型的指针 p 来指向 num 的地址。然后，通过判断当前系统的字节序来输出 num 的字节序。

如果当前系统为大端序，则按照原始顺序输出各个字节；如果当前系统为小端序，则需要逆序输出各个字节。

例如，如果当前系统为大端序，则输出结果为：

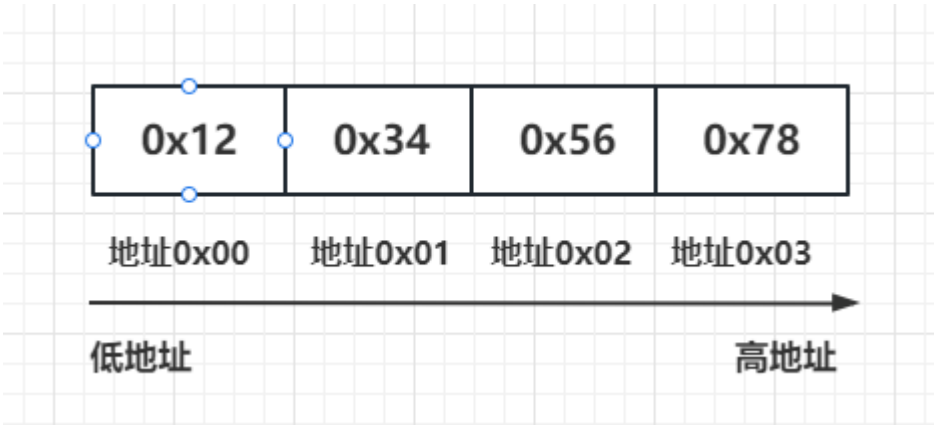
```
1. 原始数据：12345678
2. 当前系统为大端序
3. 字节序为：12 34 56 78
```

如果当前系统为小端序，则输出结果为：

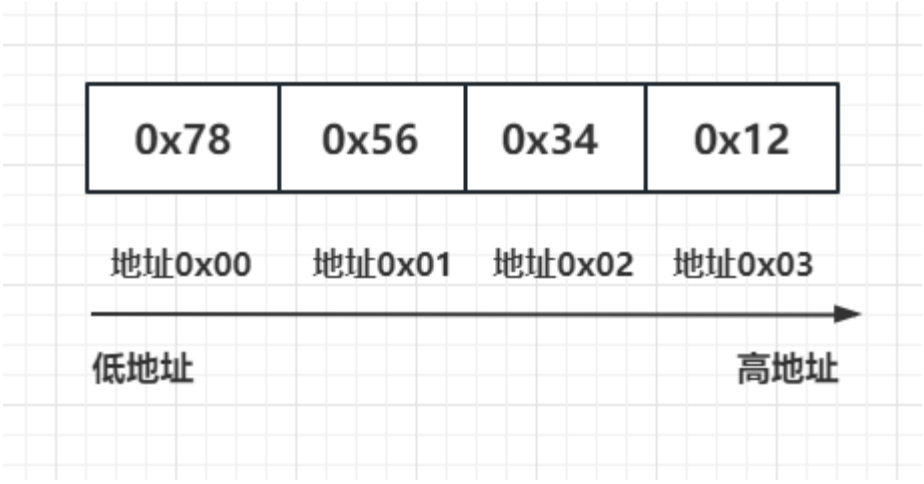
```
1. 原始数据：12345678
2. 当前系统为小端序
3. 字节序为：78 56 34 12
```

以上是 C++ 中实现大端字节序和小端字节序的示例。画个图表示大端和小端字节序区别。

大端模式



小端模式



服务器使用网络字节序

为保证字节序一致性，网络传输使用网络字节序，也就是大端模式。
在 boost::asio 库中，可以使用 boost::asio::detail::socket_ops::host_to_network_long() 和 boost::asio::detail::socket_ops::host_to_network_short() 函数将主机字节序转换为网络字节序。具体方法如下：

```
#include <boost/asio.hpp>
#include <iostream>

int main()
{
    uint32_t host_long_value = 0x12345678;
    uint16_t host_short_value = 0x5678;

    uint32_t network_long_value = boost::asio::detail::socket_ops::host_to_network_long(host_long_value);
    uint16_t network_short_value = boost::asio::detail::socket_ops::host_to_network_short(host_short_value);

    std::cout << "Host long value: 0x" << std::hex << host_long_value << std::endl;
    std::cout << "Network long value: 0x" << std::hex << network_long_value << std::endl;
    std::cout << "Host short value: 0x" << std::hex << host_short_value << std::endl;
    std::cout << "Network short value: 0x" << std::hex << network_short_value << std::endl;

    return 0;
}
```

上述代码中，使用了 boost::asio::detail::socket_ops::host_to_network_long() 和 boost::asio::detail::socket_ops::host_to_network_short() 函数将主机字节序转换为网络字节序。

host_to_network_long() 函数将一个 32 位无符号整数从主机字节序转换为网络字节序，返回转换后的结果。host_to_network_short() 函数将一个 16 位无符号整数从主机字节序转换为网络字节序，返回转换后的结果。

在上述代码中，分别将 32 位和 16 位的主机字节序数值转换为网络字节序，并输出转换结果。需要注意的是，在使用这些函数时，应该确保输入参数和返回结果都是无符号整数类型，否则可能会出现错误。
同样的道理，我们只需要在服务器发送数据时，将数据长度转化为网络字节序，在接收数据时，将长度转为本机字节序。
在服务器的HandleRead函数里，添加对data_len的转换，将网络字节转为本地字节序

```
1. short data_len = 0;
2. memcpy(&data_len, _recv_head_node->_data, HEAD_LENGTH);
3. //网络字节序转化为本地字节序
4. data_len=boost::asio::detail::socket_ops::network_to_host_short(data_len);
5. cout << "data_len is " << data_len << endl;
```

在服务器的发送数据时会构造消息节点，构造消息节点时，将发送长度由本地字节序转化为网络字节序

```
1.  MsgNode(char * msg, short max_len):_total_len(max_len + HEAD_LENGTH),_cur_len(0){
2.      _data = new char[_total_len+1]();
3.      //转为网络字节序
4.      int max_len_host = boost::asio::detail::socket_ops::host_to_network_short(max_len);
5.      memcpy(_data, &max_len_host, HEAD_LENGTH);
6.      memcpy(_data+ HEAD_LENGTH, msg, max_len);
7.      _data[_total_len] = '\0';
8.  }
```

客户端也遵循同样的处理。

消息队列控制

发送时我们会将发送的消息放入队列里以保证发送的时序性，每个session都有一个发送队列，因为有的时候发送的频率过高会导致队列增大，所以要对队列的大小做限制，当队列大

于指定数量的长度时，就丢弃要发送的数据包，以保证消息的快速收发。

```
1. void CSession::Send(char* msg, int max_length) {
2.     std::lock_guard<std::mutex> lock(_send_lock);
3.     int send_que_size = _send_que.size();
4.     if (send_que_size > MAX_SENDQUE) {
5.         cout << "session: " << _uuid << " send que fulled, size is " << MAX_SENDQUE << endl;
6.         return;
7.     }
8.
9.     _send_que.push(make_shared<MsgNode>(msg, max_length));
10.    if (send_que_size>0) {
11.        return;
12.    }
13.    auto& msgnode = _send_que.front();
14.    boost::asio::async_write(_socket, boost::asio::buffer(msgnode->_data, msgnode->_total_le
    n),
15.        std::bind(&CSession::HandleWrite, this, std::placeholders::_1, SharedSelf()));
16. }
```

总结

本文介绍了如何使用网络字节序以及为什么使用网络字节序作为网络传输，且为发送队列设置了阈值保证发送数据的高效性。
源码链接https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(2634) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(510) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)